

Stream Processing

Concepts

Design Patterns

Architecture

Frameworks

Objectives

- Concepts
- Design Patterns
- Architecture
- Frameworks

Stream Processing

- Data stream also called event stream or streaming data.
- Data stream is an abstraction representing an unbounded dataset. Unbounded means infinite and ever growing. The dataset is unbounded because over time, new records keep arriving.
 - This simple model (a stream of events) can be used to represent pretty much every business activity we care to analyze.
- Few other attributes of event streams model, in addition to their unbounded nature:
 - Event streams are ordered
 - Events, once occurred, can never be modified.
 - Event streams are replayable in order to correct errors, try new methods of analysis, or perform audits.
 - Request Response – The lowest latency paradigm around response times
 - Batch processing – The high-latency/high-throughput option
 - Stream processing – Filling the gap between the request-response world where we wait for events that take two milliseconds to process and the batch processing world where data is processed once a day and takes eight hours to complete.
 - Most business processes don't require an immediate response within milliseconds but can't wait for the next day either.

Stream-Processing Concepts

- Stream processing is very similar to any type of data processing
 - Write code that receives data
 - Do something with the data (few transformations, aggregates, enrichments, etc.).
 - Then place the result somewhere
- Few concepts related to Stream Processing Application
 - Time
 - State
 - Stream-Table Duality
 - Time Windows

Stream Processing Concepts - Time

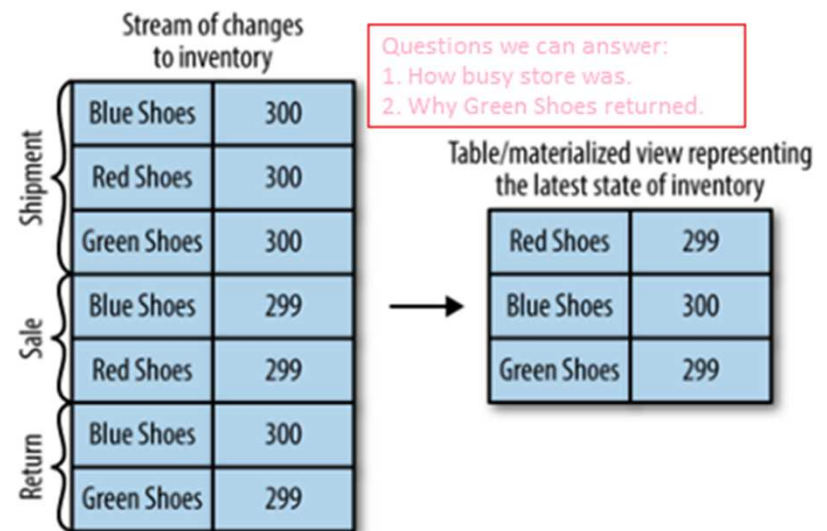
- Stream-processing systems typically refer to the following notions of time
 - Event time – time the events we are tracking occurred and the record was created
 - Log append time – time the event arrived to the Kafka broker and was stored there.
 - Processing time – time at which a stream-processing application received the event in order to perform some calculation.

Stream Processing Concepts - State

- As long as you only need to process each event individually, stream processing is a very simple activity.
 - Example find the transactions over \$10,000 and email the relevant salesperson
- Stream processing refers to several types of state:
 - Local or internal state
 - State that is accessible only by a specific instance of the stream-processing application.
 - Advantage – extremely fast
 - Disadvantage – limited to the amount of memory available.
 - Many of the design patterns in stream processing focus on ways to partition the data into substreams that can be processed using a limited amount of local state.
 - External state
 - State that is maintained in an external datastore
 - Advantage – virtually unlimited size
 - Disadvantage – extra latency and complexity introduced with an additional system.
 - Most stream-processing apps try to avoid this state.

Stream Processing Concepts - Stream Table Duality

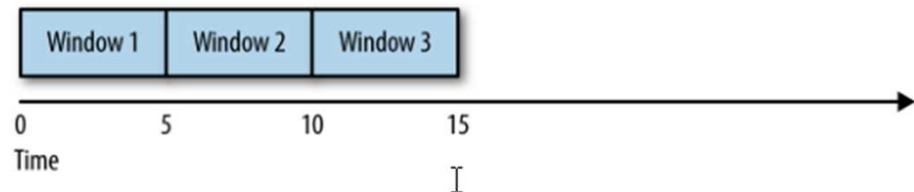
- Streams and Tables are two sides of the same coin
 - A table contains a current state of the world.
 - Streams are a string of events wherein each event caused a change
 - Systems that allow you to transition back and forth between the two ways of looking at data are more powerful than systems that support just one.
 - In order to convert a table to a stream, we need to capture the changes that modify the table. Take all those insert, update, and delete events and store them in a stream.
 - Most databases offer change data capture (CDC) solutions for capturing these changes
 - There are many Kafka connectors that can pipe those changes into Kafka where they will be available for stream processing.



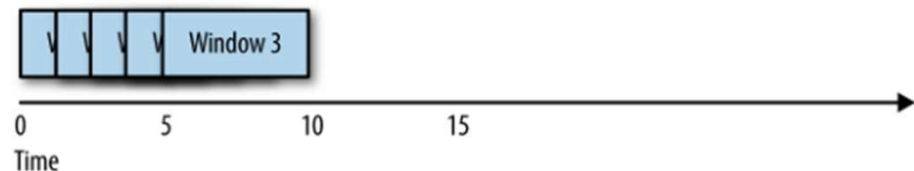
Stream Processing Concepts - Time Windows

- Most operations on streams are windowed operations
 - Operating on slices of time like moving averages, top products sold this week, 99th percentile load on the system, etc.
- For example, when calculating moving averages, we want to know
 - Size of the window
 - How often the window moves (advance interval)
 - How long the window remains updatable
- **Tumbling Window:** when advance interval is same as size of window.

Tumbling Window: 5-minute window, every 5 minutes.



Hopping Window: 5-minute window, every 1 minute.
Windows overlap, so events belong to multiple windows.

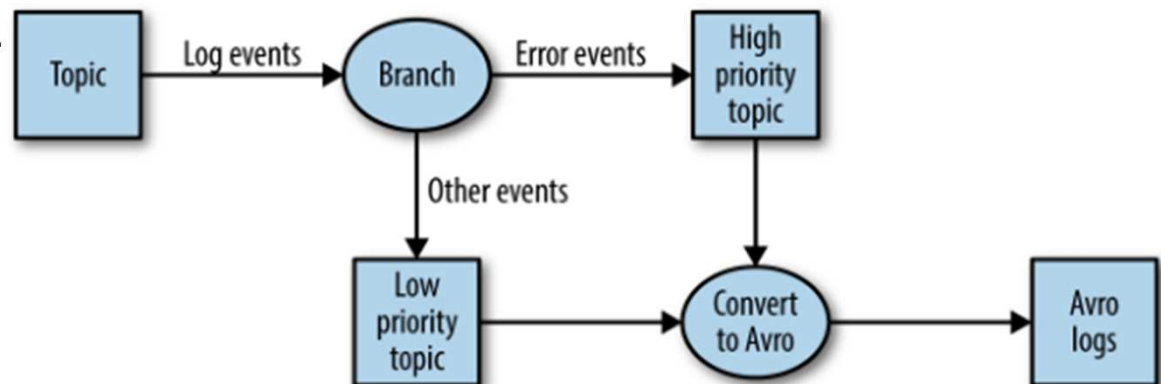


Stream-Processing Design Patterns

- There are some basic design patterns, which are known solutions to common requirements of stream-processing architectures.
- Few of those well-known patterns and show how they are used with a few examples.
 - Single-Event Processing
 - Processing with Local State
 - Multiphase Processing/Repartitioning
 - Processing with External Lookup: Stream-Table Join
 - Streaming Join
 - Out-of-Sequence Events
 - Reprocessing

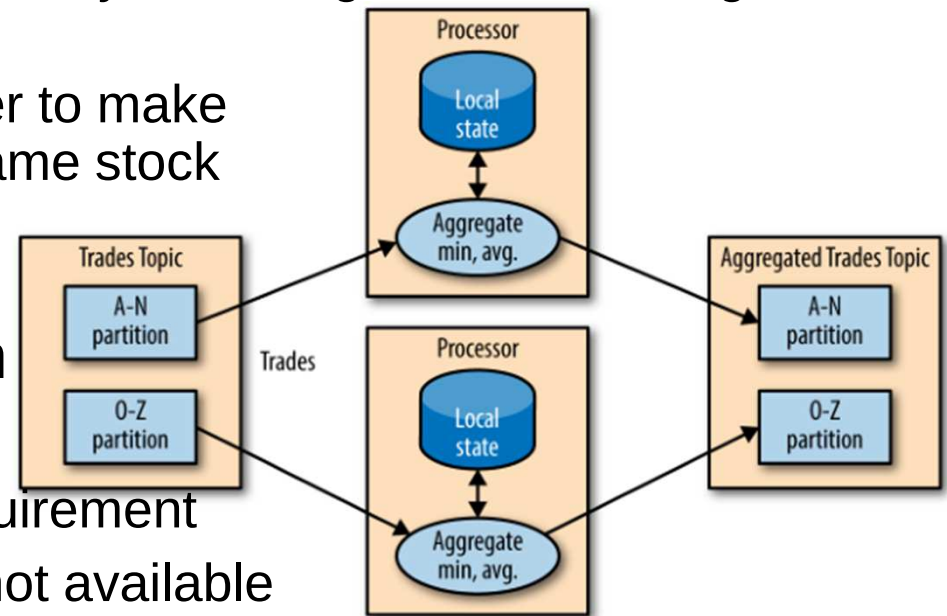
Single-Event Processing Pattern

- Most basic pattern of stream processing is the processing of each event in isolation.
- Also known as a map/filter pattern because it is commonly used to filter unnecessary events.
- In this pattern, the stream-processing app consumes events from the stream, modifies each event, and then produces the events to another stream.
 - For Example – Read log messages from a stream and writes ERROR events into a high-priority, and the rest of the events into a low-priority stream.



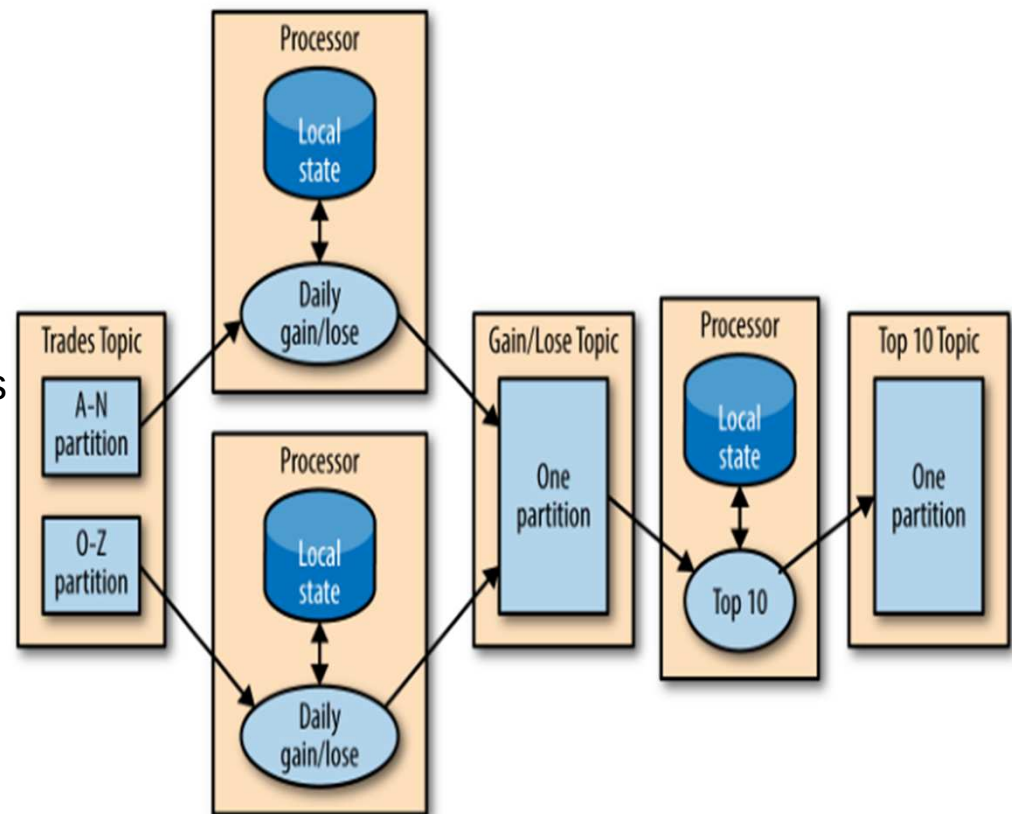
Processing with Local State Pattern

- Stream-processing applications are concerned with aggregating information, especially time-window aggregation. Great when aggregate needed.
- Aggregations require maintaining a state for the stream.
 - For Example – An example of this is finding the minimum and maximum stock prices for each day of trading and calculating a moving average.
 - We can use a Kafka Partitioner to make sure that all events with the same stock symbol are written to the same partition.
- Complexity increases when
 - Memory usage blocker is hit.
 - Persistence i.e. Retention requirement
 - Rebalancing i.e. Consumers not available



Multiphase Processing/Repartitioning Pattern

- Local state is great if you need a group by type of aggregate.
- But what if you need a result that uses all available information?
 - For example, suppose we want to publish top 10 stocks each day
- What we need is a 2-phase approach.
 - First, we calculate daily gain/loss for each stock symbol with a local state.
 - The second topic, which contains just the daily summary for each stock symbol, is obviously much smaller with significantly less traffic

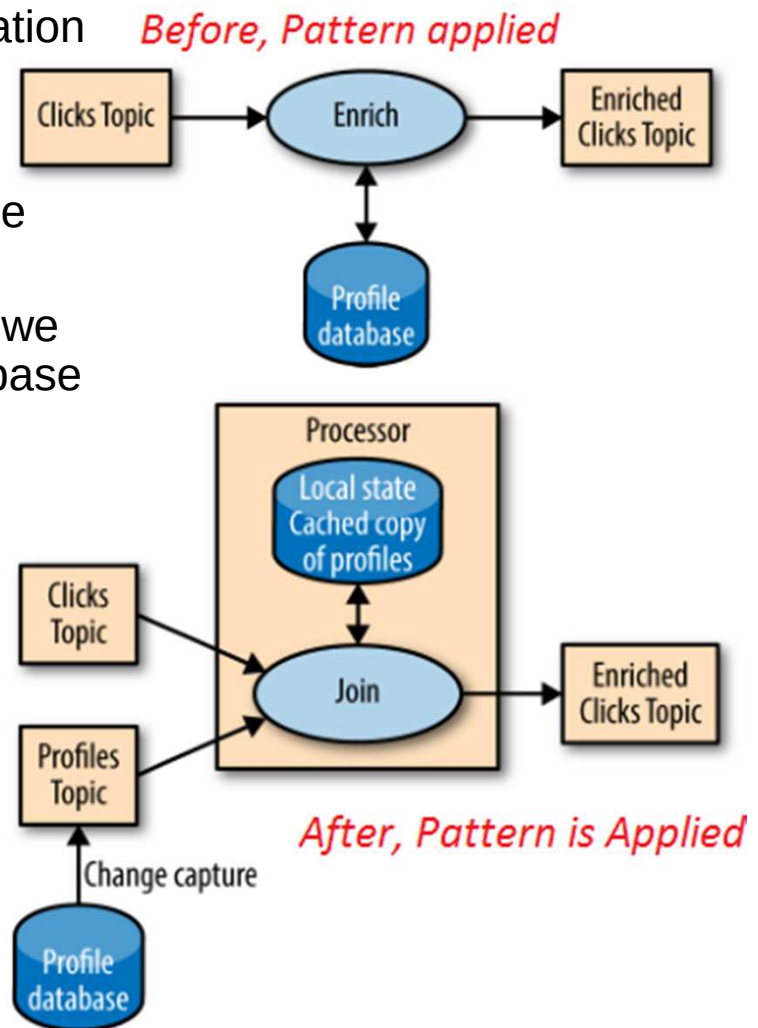


Stream-Table Join Pattern

- Stream processing requiring integration with data external to stream.
 - For Example – enriching click stream information with data about the users who clicked.

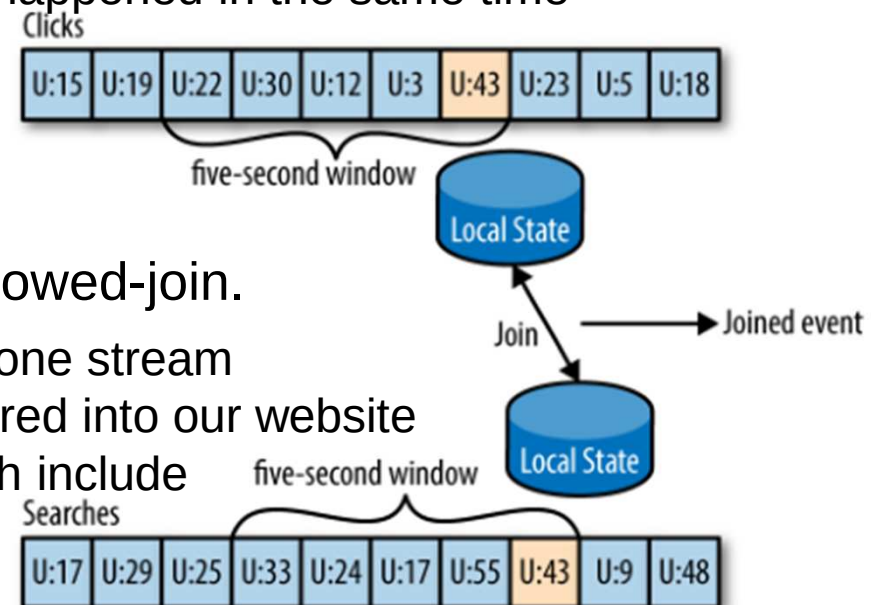
- Challenges in Stream-Table Join

- External lookup adds significant latency to the processing of every record
- In order to get good performance and scale, we need to cache the information from the database in our stream-processing application.
 - Challenge here – How frequent we should perform lookup to update our cache.
- Solution – Capturing changes to the database as events in a stream is known as CDC, and if you use Kafka Connect you will find multiple connectors capable of performing CDC and converting database tables to a stream of change events.



Streaming Join Pattern

- Sometimes you want to join two real event streams rather than a stream with a table.
 - Challenge – you are joining the entire history
 - Solution – Try to match events in one stream with events in the other stream that have the same key and happened in the same time-windows.



- Streaming-join is also called a windowed-join.
 - For example, let's say that we have one stream with search queries that people entered into our website and another stream with clicks, which include clicks on search results. We want to match search queries with the results they clicked on so that we will know which result is most popular for which query.

Out-of-sequence Events & Reprocessing

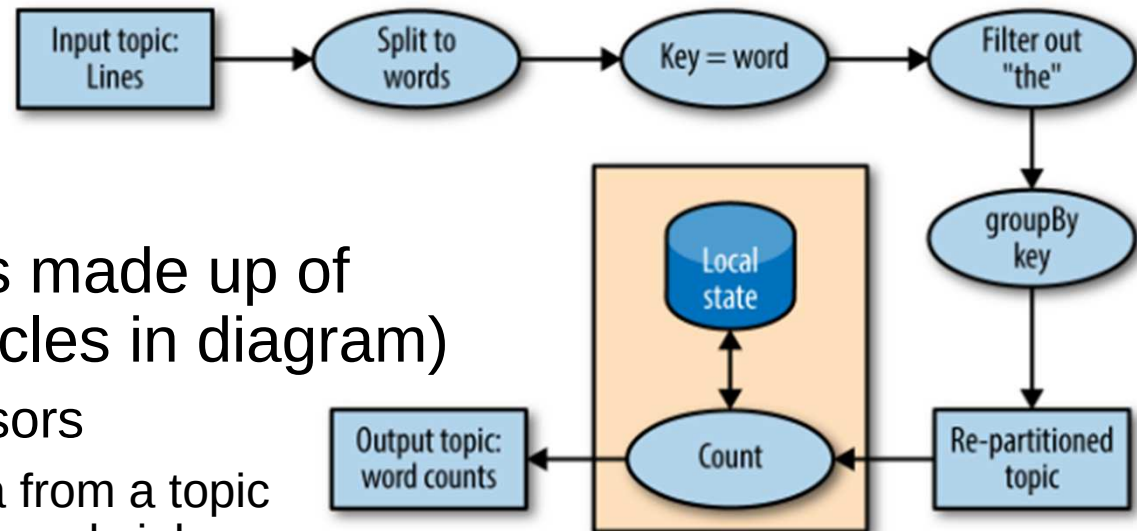
- Out-of-Sequence Events
 - Handling events that arrive at the stream at the wrong time is a challenge.
 - Out-of-sequence events happen quite frequently and expectedly in IoT. For example – a mobile device loses WiFi signal for a few hours
 - Our streams applications need to be able to handle those scenarios. This typically means the application has to do the following:
 - Recognize that an event is out of sequence
 - Define a time period during which it will attempt to reconcile out-of-sequence events.
 - Several stream-processing frameworks, including Google's Dataflow and Kafka Streams
- Reprocessing
 - Sometimes reprocessing of events is required, two variants of it
 - For example – The existing stream-processing app is buggy. We fix the bug and we want to reprocess the event stream and recalculate our results
 - Use high retention period within Kafka – recommended.
 - Kafka Streams has a tool for resetting the state for a stream-processing app

Kafka Streams: Architecture Overview

- Building a Topology
- Scaling the Topology
- Surviving Failures

Building Topology

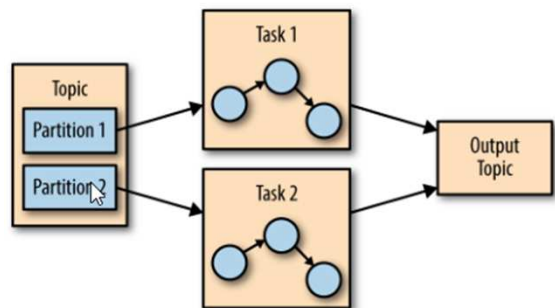
- Every streams application implements and executes at least one topology. Topology is a set of operations and transitions that every event moves through from input to output.



- The topology is made up of processors (circles in diagram)
 - Source processors
 - Consume data from a topic and pass it on, and sink processors
 - Sink processors
 - Take data from earlier processors and produce it to a topic.
 - A topology always starts with one or more source processors and finishes with one or more sink processors.

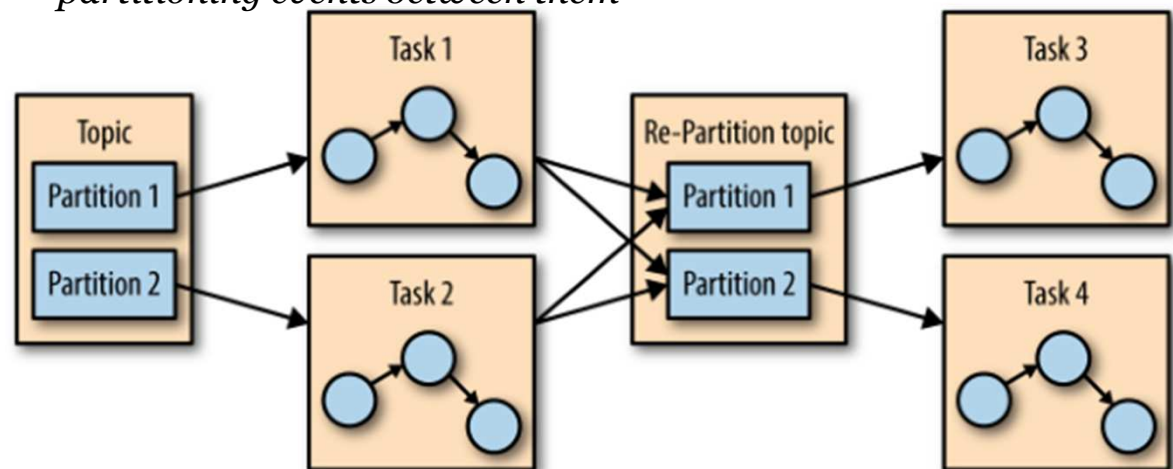
Scale the Topology

- Kafka Streams scales by allowing multiple threads of executions
- The Streams engine parallelizes execution of a topology by splitting it into tasks.



Two tasks running the same topology—one for each partition in the input topic

Two sets of tasks processing events with a topic for re-partitioning events between them



Surviving Failures

- The same model that allows us to scale our application also allows us to gracefully handle failures.
- Kafka is highly available, and therefore the data we persist to Kafka is also highly available.
- So if the application fails and needs to restart, it can look up its last position in the stream from Kafka and continue its processing from the last offset it committed before failing.

How to Choose a Stream-Processing Framework

- When choosing a stream-processing framework, it is important to consider the type of application you are planning on writing.
- Different types of applications call for different stream-processing solutions:
 - Ingest – The goal is to get data from one system to another
 - Reconsider whether you want a stream processing system or a simpler ingest-focused system like Kafka Connect.
 - Low milliseconds actions – Any application that requires almost immediate response.
 - Request-response patterns are often better suited to this task.
 - Asynchronous microservices – These applications may need to maintain a local state caching events as a way to improve performance.
 - Need stream processing system that integrates well with your message bus of choice , has change capture capabilities, good support of a local store.
 - Near real-time data analytics – These streaming applications perform complex aggregations and joins
 - Stream-processing system with great support for a local store, not for maintenance but for aggregations
- Some examples available
 - Apache Storm – It is continuing to be a leader in real-time analytics (from tutorialpoint.com)
 - Apache Spark – Built on top of Hadoop MapReduce (from wiki)
 - ...

Summary

- Concepts
- Design Patterns
- Architecture
- Frameworks